



SPAC: Automating FPGA-based Network Switches with Protocol Adaptive Customization

Guoyu Li[†], Yang Cao[†], Lucas H L Ng[†], Alexander Charlton[†], Qianzhou Wang[†], Will Punter[†], Philippos Papaphilippou[‡], Ce Guo[†], Hongxiang Fan[†], Wayne Luk[†], Saman Amarasinghe[§] and Ajay Brahmakshatriya[§]
[†] Imperial College London, [‡] University of Southampton, [§] Massachusetts Institute of Technology
 {g.li25, yang.cao24, lucas.ng22, alexander.charlton22, qianzhou.wang17, will.punter23, c.guo, hongxiang.fan, w.luk}@imperial.ac.uk; P.Philippos@soton.ac.uk; saman@csail.mit.edu; ajaybr@mit.edu

Abstract—With network requirements diverging across emerging applications, latency-critical services demand minimal logic delay, while hyperscale training and collectives require sustained line-rate throughput for synchronized bulk transfers. This divergence creates an urgent need for custom network switches tailored to specialized protocols and application-specific traffic patterns. This paper presents SPAC (Switch and Protocol Adaptive Customization), a novel approach that automates the generation of FPGA-based network switches co-optimized for custom protocols and application-specific traffic patterns. SPAC introduces a unified workflow with a domain-specific language (DSL) for protocol-architecture co-design, a library of modular HLS-based adaptive switch components, and a trace-aware Design Space Exploration (DSE) engine. By providing a multi-fidelity simulation stack, SPAC enables rapid identification of Pareto-optimal designs prior to deployment. We demonstrate the efficacy of the domain-specific adaptation of SPAC across a spectrum of real-world scenarios, spanning from latency-sensitive sensor and HFT networks to hyperscale datacenter fabrics. Experimental results show that by tailoring the micro-architecture and protocol to the specific workload, SPAC-generated designs reduce LUT and BRAM usage by 55% and 53%, respectively. Compared to fixed-architecture counterparts, SPAC delivers latency reductions ranging from 7.8% to 38.4% across various tasks while maintaining adequate resource consumption and packet drop rate.¹

I. INTRODUCTION

Modern network requirements have diverged sharply. Real-time systems (e.g., High Frequency Trading) demand ultra-low latency with minimal logic delay, while hyperscale AI training requires maximum throughput for synchronized bulk transfers [1]. A fundamental hardware trade-off makes it difficult for a single switch architecture to serve both. Increasing switch capability, such as adding deep buffering or complex scheduling, inevitably increases logic latency and resource usage, while general-purpose switches using fixed micro-architectures fail to adapt to diverse workloads. This architectural rigidity leaves significant performance on the table. As shown in Figure 1 (left), switch architectures are highly sensitive to traffic patterns: with iSLIP-based design [2] favoring uniform traffic and EDRRM-based design [3] handling bursts better. Therefore, it is essential to customize the switch architecture for different scenarios.

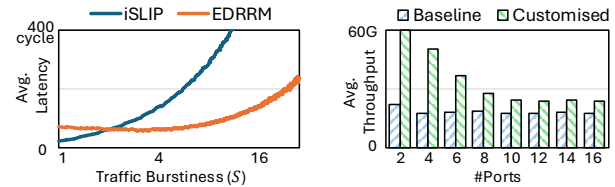


Fig. 1. Hardware Sensitivity (left): different scheduler architectures favor different traffic patterns. Protocol Sensitivity (right): throughput comparison of a SPAC switch using a standard protocol versus a custom protocol.

Beyond hardware logic, the transport protocol itself also matters. General-purpose protocols (e.g., Ethernet/IP) often impose unnecessary headers and processing overheads for specialized workloads. Commodity devices rigidly couple fixed hardware with standard parsers, lacking the flexibility to strip away distinct protocol layers or adjust logic for custom flows. While the emergence of P4 [4] allows for protocol customization in switches, their Protocol Independent Switch Architecture (PISA) designs limit the feasibility of improving performance by tweaking the switch architecture. As shown in Figure 1 (right), by removing generic protocol overheads and tailoring the pipeline to the flow, the custom design achieves higher throughput. These results confirm that true optimization requires co-designing both the switch architecture and the protocol.

However, developing such co-design custom systems remains prohibitively difficult due to three critical challenges.

Challenge-1: Fixed Architectures versus Diverse Applications. Co-optimizing the switch architecture with the protocol is essential, yet current solutions either lack architectural reconfigurability or remain tightly coupled to standard protocols. For instance, emerging applications often require specialized processing logic beyond standard forwarding, developers may need to inject custom compute kernels alongside custom protocols to maximize performance. However, fixed architectures (e.g., PISA) cannot handle complex stateful logic (e.g., floating-point aggregation).

Challenge-2: Efficient Simulation for Custom Protocol Network Switches. An application's specific features can cause the switch to perform below its theoretical potential. Simulation in real-world traffic is crucial to identify bottlenecks, but current software simulations (e.g., ns-3 [5]) lack native support for custom protocols and custom switch devices.

¹Our design is open source at: <https://github.com/spac-proj/SPAC>

Challenge-3: The Coupling Trap of Vertical Integration.

Optimizing a network for a specific application requires tight coupling between the protocol format and the switch micro-architecture. A small change in the application layer can ripple across the entire hardware stack, requiring developers to manually rewrite RTL parsing logic and re-tune architectural parameters. This cross-layer dependency makes it practically impossible to keep the network hardware synchronized with agile software iterations.

To address these challenges, we introduce **SPAC**, Switch and Protocol Adaptive Customization. SPAC bridges the gap between protocol definition and line-rate hardware generation using an HLS-based dynamic configurable switch template with hardware-calibrated design space exploration (DSE), enabling the automatic deployment of optimized networking substrates tailored for both performance and resources. Figure 2 presents the end-to-end workflow.

Our main contributions are as follows.

- **Adaptive Network Switch Architecture:** To solve **Challenge-1**, we designed a high-performance modular HLS-based switch design that provides a custom-protocol parser, multiple forwarding structures, scheduling algorithms, and buffer allocation strategies. SPAC switch also provides the architectural hooks and interfaces to enable custom logic injection.
- **Efficient Custom Network-Stack Simulator:** To address **Challenge-2**, we created a multi-granularity network simulation system supporting both statistical and hardware-aware modeling. It enables the verification of generated switches and custom protocols on ns-3 [5], providing rapid performance insights prior to synthesis.
- **SPAC Domain-Specific Language (DSL) and Two-Stage Custom Network Workflow:** We propose a two-stage workflow based on the SPAC DSL to streamline the development of programmable switches and tackle **Challenge-3**. By generating protocol drivers and HLS parsing libraries that link to switch behaviors via semantic binding, SPAC effectively decouples protocol specifications from the core switching logic. Furthermore, the framework enables automated DSE by integrating multi-granularity simulation with a hardware resource model. This allows for the rapid identification of Pareto-optimal designs, balancing latency and throughput trade-offs under strict hardware constraints.

II. BACKGROUND AND RELATED WORK

A. Programmable Network Devices

A network switch comprises four fundamental components: a packet parser, a forward table, packet buffers, and a switching fabric (scheduling algorithms). Existing FPGA-based research typically focuses on optimizing these individual modules in isolation. For instance, platforms like GCQ [6], Hipernetch [7], SMiSLIP [8], NetFPGA [9] and CusComNet [10] optimize the reconfigurable fabric layout. Other works focus on specific sub-problems, such as adapting

iterative scheduling algorithms like iSLIP [2] to meet FPGA timing constraints [11], [8]. While techniques like hierarchical design [6] and resource sharing improve local performance, these efforts are usually specific to particular architectures, lack a unified framework that allows developers to flexibly compose these optimized modules into a complete, customized switch.

A separate line of work focuses on programmable NIC and SmartNIC platforms. OpenNIC [12] and Corundum [13] provide open-source FPGA NIC shells, while commercial DPUs (e.g., NVIDIA BlueField) offer fixed offload pipelines. eBPF-based approaches such as eBPFflow [14] and Nanotube [15] enable programmable packet processing on FPGA NICs within fixed pipeline structures. PANIC [16] explores flexible multi-tenant NIC architectures, Pigasus [17] demonstrates FPGA-based network security at 100 Gbps, and FlowBlaze [18] introduces stateful packet processing abstractions. Recent efforts also investigate SmartNIC datapath acceleration [19], software packet processing on FPGA NICs [20], and hardware offloading for virtual switches [21]. These systems primarily accelerate *endpoint* functions (e.g., host offloads, protocol termination) rather than switch-level contention and queueing under multi-host traffic, which is the focus of SPAC.

In parallel to structural optimizations, researchers actively explore in-Network computing (INC) to offload application-specific computation. FPGAs serve as an ideal platform for this domain due to their flexibility [22]. Recent studies demonstrate effective acceleration for distributed workloads, such as gradient aggregation for machine learning [23] and dynamic computation offloading [24]. However, these implementations often operate as isolated point solutions. They typically embed custom logic within rigid pipelines or require significant manual effort to comply with infrastructure limitations [25], [26]. This tight coupling compels developers to manually re-implement basic switching infrastructure for every new application [27]. This limitation underscores the critical need for a modular, fully synthesizable switch architecture that can rapidly integrate custom computing kernels when application needs.

B. Custom Network Protocols

While Ethernet and TCP/IP [28], [29] provide broad interoperability, their fixed header structures introduce substantial redundancy. Consequently, modern high-performance domains have adopted specialized protocols to meet stringent targets. For instance, RoCEv2 [30] for AI gradient synchronization, FIX [31] and FAST [32] for ultra-low-latency trading, PROFINET [33] and EtherCAT [34] for industrial control, and DCTCP [35] for data center traffic. Custom protocols can bring significant benefits to special scenarios. For example, underwater acoustic sensor networks often transmit payloads as small as 2 bytes [36]. Wrapping these in standard headers results in a standard protocol that would take at least 42B, severely limiting goodput.

C. DSLs for Network

There are many DSLs to facilitate custom protocol development. For example, P4 [4] is a data-plane language supported by some commercial switches. Compilers such as VitisNetP4 [37], P4THLS [38], P4-to-FPGA [39], and earlier efforts [40], [41], [42] map P4 descriptions to FPGA logic, while configurable parser architectures [43] target wire-speed throughput for arbitrary protocols. However, these tools primarily target data-plane parsing and match-action pipelines rather than complete switch architectures, and their reliance on fixed-stage pipeline structures limits global module configurability and stateful in-network computing capabilities. Another example is NetBlocks [36], which focuses on minimizing the overhead of custom protocol layout and generate high-performance system drivers. However, a software stack alone remains insufficient, as comprehensive protocol design requires validation through large-scale simulations and underlying hardware support.

III. SPAC OVERVIEW AND SYSTEM

Figure 2 presents the overall architecture of SPAC, which comprises three key components.

SPAC Switch Template. We abstract the switch architecture into 6 parts: *Parser* (ingress and deserialize bitstreams), *Custom Kernels*, *Forward Table* (ingress and forwarding table lookup), *VOQ Buffer* (buffer management and organization), *Scheduler* (schedule different traffic to prevent HoL or packet loss) and *Deparser* (egress and serialize packet). SPAC provides diverse hardware modules for each stage that adhere to a unified *Meta+Data* I/O stream interface, enabling modular design composition tailored to specific application and throughput requirements (Section III-B).

NS-3 Based and Statistical Network Simulator. Existing large-scale network simulators primarily support standard protocols and lack native support for custom protocols. We introduce multi-level simulation specifically designed to model custom protocol systems (Section IV-A). The *NS-3 Based Simulator* integrates our custom protocol adaptation layer into the NS-3 backend, and adds a SPAC switch device to provide more realistic network simulation. Meanwhile, the *Statistical Surrogate Model* simulates packet arrival based on switch pipelines, end-to-end latency, and buffering to quickly obtain theoretical results.

SPAC DSL and DSE Optimizer. The SPAC DSL allows high-level descriptions of custom packet structures and device behavior, enabling users to focus on application-layer design without low-level hardware details. To rapidly explore a vast design space, our DSE employs multi-level simulation models to evaluate trade-offs between traffic patterns, device scale, and logic latency. This process identifies Pareto-optimal configurations and generates the final hardware and software stack (Section IV-B).

A. The SPAC Specification Language

Unlike prior approaches that require disjoint specifications for software drivers, hardware logic, and simulation models,

SPAC serves as a single source of truth. The DSL decouples the logical definition of the network protocol from its physical realization in the FPGA fabric, enabling automated DSE. As shown in Figure 2 (left), the language primitives are categorized into three abstraction layers: Custom Protocol Definition, Semantic Binding, and Architecture Configuration.

Custom Protocol Definition. SPAC uses NetBlocks-compatible syntax[36] to specify custom protocols. By supporting bit-level serialization, NetBlocks facilitates the generation of highly optimized, compressed protocols. Additionally, by integrating the underlying high-performance network driver stack, it offers a stable ABI across simulation and real-world deployment, effectively streamlining the development workflow.

Semantic Binding. Each user-defined protocol bit-field has a semantic alias, which is mapped to the switch architecture during Semantic Binding. As shown in Figure 2, the protocol field designated for routing (i.e., the `routing_key`) must be specified, while other fields remain optional. During the header compilation stage, the SPAC Compiler locates these fields within the protocol via key-value matching and generates inlined parsing logic into the HLS header file (`packet.hpp`) for both the user kernel and the switch code. This decoupling allows users to modify the protocol layout without altering the underlying HLS implementation. By leveraging template metaprogramming, the system efficiently generates hardwired logic during switch synthesis, enabling line-rate packet parsing.

Architecture Configuration. A key innovation of SPAC is its support for different switch architectures. Instead of forcing developers to write rigid HLS code, the DSL allows the specification of architectural policies (e.g., BufferPolicy, HashPolicy) as either explicit values or Auto. When policy is set to Auto, the SPAC DSE engine infers the optimal micro-architecture selections based on traffic characteristics and network topology. To support application-specific logic beyond standard forwarding, SPAC also provides an injection point for custom HLS kernels. To incorporate these kernels into the DSE loop, users specify latency and resource boundaries via a performance interface.

B. Configurable Switch Architecture

To support the combination of diverse network fabrics, the SPAC switch design adopts a decoupled, stream-based modular design. We standardize inter-module communication using a dual-stream interface: a standard AXI-Stream for payload propagation and a compiler-synthesized MetaData side-channel for header (e.g., extracted routing keys, QoS). By encapsulating architectural state within the MetaData stream, the design ensures strict isolation between stages. For instance, the *Forward Table* can be seamlessly swapped between a FullLookUpTable array and a MultiBank Hash table without affecting the downstream Scheduler, while custom computing kernels can be injected post-parsing with zero glue logic.

1) *Protocol-Aware Parser:* To achieve line-rate processing, SPAC abandons the traditional “programmable parser” archi-

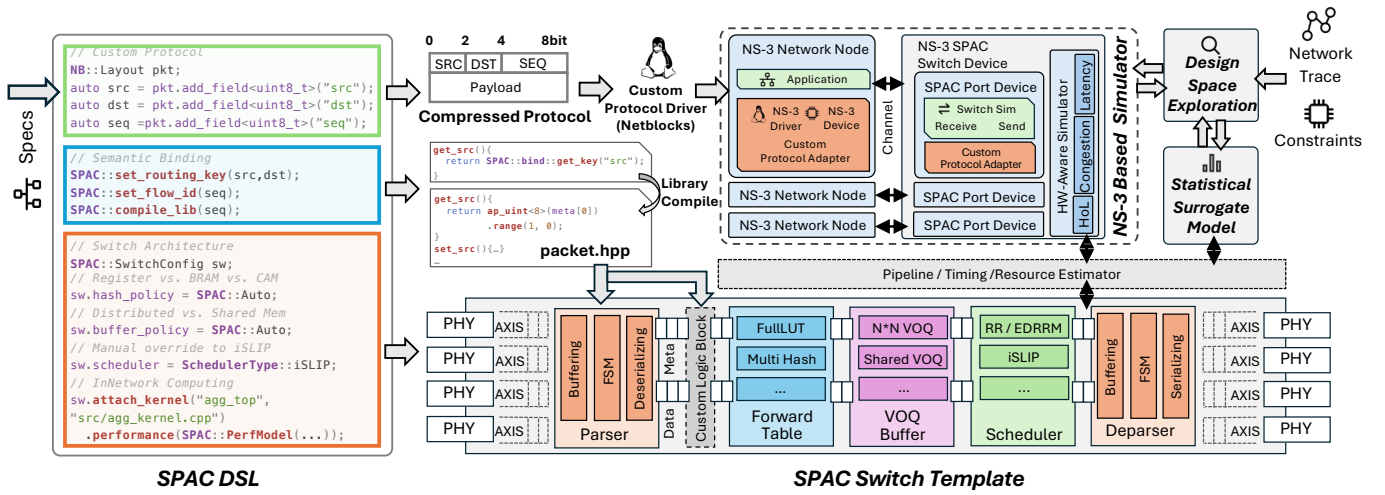


Fig. 2. SPAC System Overview.

texture (which often relies on TCAMs or runtime configuration registers) in favor of a template-driven synthesis approach. The core of our parsing subsystem is a generic HLS template, which remains completely agnostic to specific protocol details until compilation. During the synthesis phase, the SPAC compiler lowers the high-level user-defined protocol specification into a C++ header. By instantiating the HLS parser template with these generated traits, we use C++ metaprogramming to recursively compute the exact bit-offset of every field relative to the AXI-Stream flit boundaries at compile time. This mechanism automatically detects fields that straddle word boundaries to synthesize minimal state retention logic only when strictly necessary, while lowering intra-flit field accesses into hard-wired bit-slicing operations. Consequently, the resulting hardware achieves efficiency comparable to hand-optimized RTL while retaining the flexibility of high-level definitions.

2) **Forward Table**: The Forward Table maintains address-port mappings: it queries the destination address to determine the output port (or broadcast) and learns the source address on every arrival. The core optimization lies in choosing a data structure that enables multiple ports to access the table in minimal clock cycles. We implement two forwarding table variants:

The array-based **Full Lookup Table** utilizes a one-dimensional table where the address field serves as the direct index. We fully partitioned the table, and it can support simultaneous reads and writes from multiple ports in a single cycle. While highly efficient and logic-light for short addresses, such as NetBlocks' shrunk protocol, which is used in underwater robots communication, it is unsuitable for long addresses as memory usage increases exponentially. To handle longer address fields, we also provide **Multi-Bank Hash Table**, as shown in Figure 3, which uses two-dimensional tables with hash functions for indexing. The table is partitioned into multiple banks so that each port's input ideally maps to a distinct bank, minimizing conflicts. Although this allows for larger address spaces, it introduces tradeoffs in the form of additional logic for hash calculations and conflict resolution.

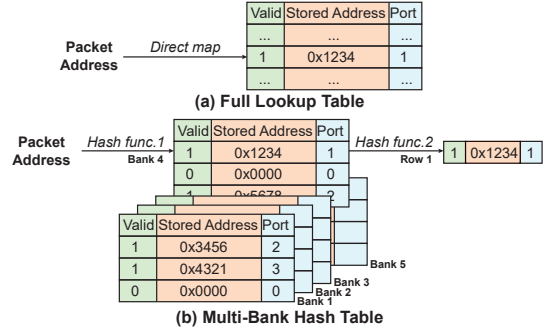


Fig. 3. Forward Table Architectures

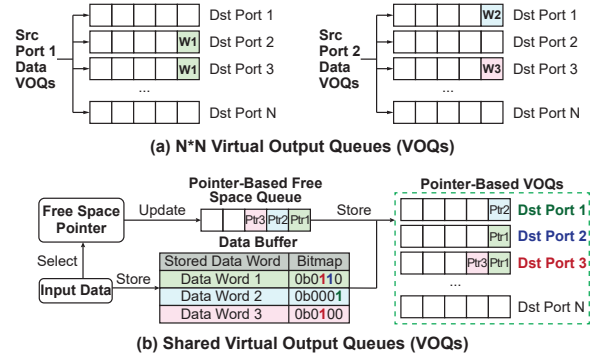


Fig. 4. N*N Virtual Output Queues (VOQs) versus Shared VOQs.

3) **Virtual-Output-Queue Buffer**: The **VOQ Buffer** is vital for decoupling input and output traffic. After the packet's destination port is labeled, it is stored into FIFO data queues based on source and destination port information. To achieve high throughput, each port maintains its own data queue to process packets in parallel. However, a single queue suffers from Head-of-Line (HoL) blocking, where a congested destination port prevents other ports from being served. To address this, we design two type of Virtual Output Queues (VOQs), as illustrated in Figure 4.

N*N Data VOQs is straightforward: packets labeled with a destination port go to the corresponding queue; broadcast

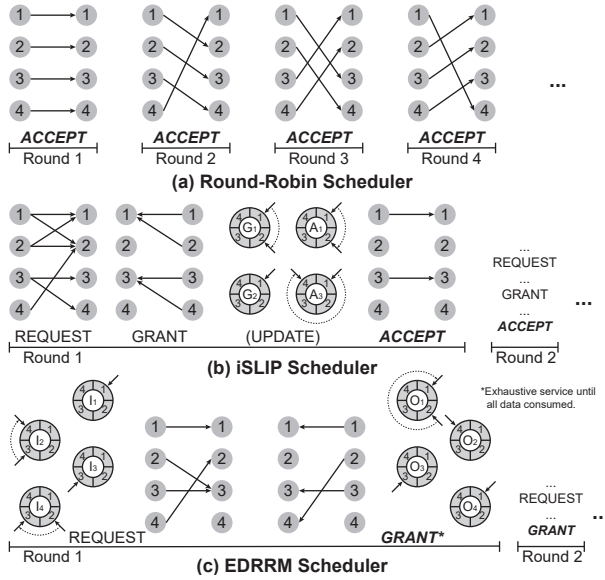


Fig. 5. RR / EDRRM / iSLIP Scheduler.

packets are copied and stored in all queues associated with the source port. The queues are fully partitioned to support reading and writing in parallel. One drawback of this implementation is that it suffers from limited FIFO depth and high memory/time costs due to data duplication for broadcasting. As a compromise, we also provide **Shared VOQs** [8], which use a central data buffer with pointer-based queues: instead of copying packet data, it stores each packet once and replicates only its pointer, using a bitmap to track pending destinations. This offers memory efficiency but introduces logic overhead for pointer management, which may impact performance.

4) *Scheduler*: The scheduling algorithm is critical for arbitrating access between input and output ports. Its primary objectives are to maximize throughput by optimizing the number of input-output matchings per cycle and to ensure fairness, preventing any source or destination port from starvation. SPAC supports three scheduling architectures: **Round-Robin** uses a simple cyclic priority rotation. It is hardware-efficient but its combinational logic latency grows with port count, requiring up to N cycles in the worst case. **iSLIP** [2] employs an iterative three-phase (*Request*, *Grant*, *Accept*) process with independent rotating pointers. While it theoretically achieves 100% throughput, the “Find-First” operation creates long combinatorial paths that often become the FPGA timing bottleneck. To address this, **EDRRM** simplifies the matching to a two-phase *Request* and *Grant* process combined with an exhaustive service strategy, reducing arbitration overhead while maintaining high efficiency for bursty traffic.

5) *Support for User-Defined Logic*: SPAC supports extensibility through a modular architecture paired with an exported HLS protocol header library. This empowers users to link custom kernels directly into the switch pipeline, utilizing our library to simplify protocol parsing. This combination lays a concrete foundation for future in-network computing research by significantly lowering the development barrier for custom

hardware acceleration.

IV. DESIGN SPACE EXPLORATION

SPAC comes with a system of tools for enumerating the custom protocol design space. They allow for the efficient simulation of custom networks, the discovery of the optimal switch architecture for a given protocol and finally RTL accurate network simulations. This system is designed to enable rapid experimentation with custom protocols and switch architectures, before commitment to compute-intensive cycle-accurate analysis is required.

A. Multi-level Simulation Model

1) *Hardware-Aware ns-3 Based Switch Simulator*: We present a flexible switch simulation architecture built upon the well-known NS-3 backend. The NS-3 network simulator contains four-layer network node abstractions: application layer, host network stack layer, Ethernet device layer, and channel layer. As shown in Figure 2, by abstracting the conventional Ethernet layer into a generalized *Custom Protocol Adapter*, our design seamlessly integrates DSL-compiled drivers (handling logic such as parsing and retransmission) thereby transcending the limitations of standard Ethernet models. This architecture encapsulates protocol state to enable multi-instance concurrency, ensuring fidelity to the specification.

On the switch side, SPAC employs a Hardware-Aligned Modeling approach. We abstract switch ports as specialized NS-3 nodes (SPAC Port Device) and explicitly model internal behaviors such as forwarding table lookups, packet buffering, and scheduling in a unified simulator. A distinguishing feature of SPAC is its support for **Hardware Back-Annotation**: by injecting performance metrics from physical FPGA experiments into the model as processing parameters, we ensure the simulator reflects realistic hardware constraints. Users can choose to enable back-annotation for high-fidelity evaluation of latency and resources, or disable it for rapid functional testing, thus balancing simulation accuracy with performance.

2) *Statistical Simulator*: To circumvent the prohibitive cost of cycle-accurate simulations for large-scale traces, we incorporate a **Statistical Surrogate Model** within the DSE loop. This model exploits the inherent determinism of FPGA-based switching logic. Specifically, the fixed Initiation Interval (II) and predictable pipeline latency of HLS modules. Instead of simulating signal-level transitions, we abstract the switch datapath into a lightweight, event-driven transaction model. By parameterizing this model with static hardware attributes (e.g., bus width, arbitration latency, and pipeline depth), our engine can process traces in seconds.

This approach enables a rapid yet accurate estimation of critical micro-architectural metrics prior to synthesis. The model estimates line-rate feasibility, BRAM lower bounds from peak VOQ occupancy, and latency distributions by combining deterministic pipeline delays with dynamic queuing effects.

To validate the fidelity of this surrogate model, we cross-verified its predictions against post-synthesis timing reports

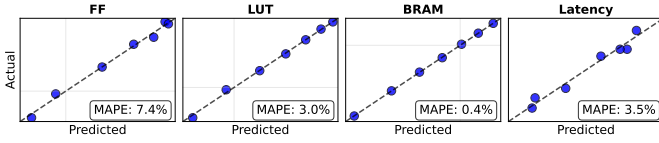


Fig. 6. Variance Analysis of Resource and Performance Estimates

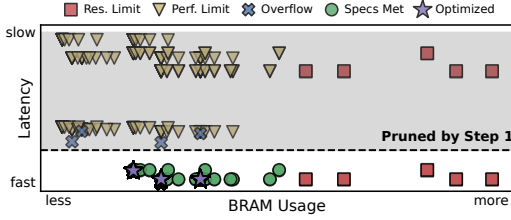


Fig. 7. DSE Algorithm Search Space Visualization.

from Vitis HLS. Figure 6 shows our surrogate model and real-world design evaluation of the resource and latency aspects of a 2-8 port design. The result with Mean Absolute Percentage Error (MAPE) of 0.4%~7.4% confirms that our lightweight abstraction effectively captures the micro-architectural impact, providing a trustworthy basis for the DSE engine.

B. DSE Algorithm

To navigate the design space efficiently, SPAC adopts a *Progressive Constraint Satisfaction* strategy. It can gradually increase the simulation granularity while reducing the search space. Algorithm 1 details this process.

The DSE engine characterizes the input trace $\mathcal{T}$ into a feature vector $\mathbf{f} = [\mathcal{I}_{burst}, \mathcal{H}_{addr}, S_{min}]$. Where $\mathcal{I}_{burst}$ shows the Index of Dispersion for Counts (IDC), acting as a congestion proxy. $\mathcal{H}_{addr}$ is the entropy of destination addresses, indicating the effectiveness of caching, and S_{min} shows the minimum packet payload observed in the windowed trace. This metric defines the worst-case arrival rate and sets the strict timing budget for the pipeline.

We apply hardware constraints with tolerance margins to prune architecturally infeasible candidates while preserving designs that may recover performance through compiler optimizations or resource over-provisioning. For an architecture template a with an initiation interval II_a , we introduce a timing relaxation factor δ and make sure $T_{proc} > (1 + \delta) \cdot T_{arrival}$, where $T_{proc} = II_a / F_{clk}$ and $T_{arrival} = (S_{min} \times 8) / \text{LinkRate}$. For the surviving candidates, we execute a One-Shot *Ideal* Simulation with infinite buffer constraints. We record the maximum queue occupancy (Q_{max}) and the latency distribution for every port in the trace. If a design point violates the 99th-percentile latency SLA even with infinite buffering, it will be dropped. Next, we explore the optimal VOQ buffer size based on statistical guarantees. Using the queue occupancy histogram collected in Stage 2, we identify the specific queue depth d_{opt} corresponding to the target tail drop rate ϵ . We then map d_{opt} to physical FPGA resources by aligning it with the switch data width. Candidates that violate the total BRAM capacity constraint are pruned. Finally, the engine runs a verification simulation with the derived parameters to ensure the discretized sizing meets all SLAs.

Algorithm 1: Progressive Constraint Satisfaction DSE

Input : Trace $\mathcal{T}$, Templates $\mathcal{A}$, Constraints $\mathcal{C}_{SLA}, \mathcal{C}_{Res}$
Output: Optimal Configuration $\mathbf{x}^*$

```

// Stage 1: Static Pruning
 $\mathbf{f} \leftarrow [\mathcal{I}_{burst}, \mathcal{H}_{addr}, S_{min}] \leftarrow \text{Analyze}(\mathcal{T})$ ;
 $\mathcal{A}_{active} \leftarrow \mathcal{A}$ ;
foreach  $a \in \mathcal{A}_{active}$  do
     $T_{arrival} \leftarrow (S_{min} * 8) / \text{LinkRate}$ ;
     $T_{proc} \leftarrow a.II / \text{Freq}$ ;
    if  $T_{proc} > (1 + \delta) * T_{arrival}$  then
         $\mathcal{A}_{active}.remove(a)$ ;
    end
end
// Stage 2: Coarse-grained Profiling
 $\mathcal{A}_{valid} \leftarrow \emptyset$ ;
foreach  $a \in \mathcal{A}_{active}$  do
    // Infinite Buffer Simulation
     $Q_{hist}, L_{dist} \leftarrow \text{Run\_Surrogate\_Model}(a, \mathcal{T})$ ;
    if  $\text{Percentile}(L_{dist}, 99) \leq \mathcal{C}_{SLA}.Latency$  then
         $\mathcal{A}_{valid}.add(\{a, Q_{hist}, L_{dist}\})$ ;
    end
end
// Stage 3: Statistical Sizing
 $\mathbf{x}^* \leftarrow \text{NULL}$ ;
foreach  $\{a, Q_{hist}\} \in \text{TopK Latency}(\mathcal{A}_{valid})$  do
     $d_{align} \leftarrow \text{AlignToBRAM}(Q_{hist}, \text{Width})$ ;
    if  $\text{TotalBRAM}(d_{align}) \leq \mathcal{C}_{Res}$  then
         $\mathbf{x}_{curr} \leftarrow \{a, d_{align}\}$ ;
        if  $\text{Run\_NS3\_Sim}(\mathbf{x}_{curr})$  meets  $\mathcal{C}_{SLA}$  then
             $\text{UpdateOptimal}(\mathbf{x}^*, \mathbf{x}_{curr})$ ;
        end
    end
end
return  $\mathbf{x}^*$ 

```

To validate, we designed a brute-force enumeration. Figure 7 plots the primary resource usage (BRAM) and latency for Incast small-packet bursts across all combinations of architectures and buffer sizes. The design points identified by SPAC DSE (marked with $\star$) lie on the Pareto-optimal frontier: the first stage prunes infeasible architectures, and the trace-aware buffer allocation then locates the resource-minimal solution.

V. EVALUATION

A. Experiment Setup

Hardware. We implement our design using Vitis HLS 2023.2 and synthesize for an AMD Alveo U45N Network Accelerator Card (xcu26-vsua1365-2LV-e), hosted on a server equipped with an AMD EPYC 9335 32-core processor. The target clock period is set to 350MHz.

Workloads. We employ multiple traffic traces:

- **Industry:** Sources real-world SCADA operations from the Dataset of SCADA traffic captures from a medical waste incinerator [44].
- **High-Frequency Trading (HFT):** generated based on traffic patterns from a real-world HFT Company [45], featuring low latency and high burstiness.
- **RL(All-Reduce):** Simulates traces in distributed reinforcement learning based on iSwitch [46], showing regularity and hotspots.
- **DataCenter:** Models microservice dependencies from the Alibaba Cluster Trace [47] and randomly deploy them to 8 nodes to simulate Kubernetes-based containerization.

TABLE I
COMPARISON OF RELATED WORK: FUNCTIONS, IMPLEMENTATION, RESOURCE OVERHEAD, AND PERFORMANCE.

Configuration			Function				Resources				Performance		
Switch	Design Language	Width (bits)	Ports	Parser	Fwd Table	Scheduler	VOQ Buffer	LUT (K)	FF (K)	BRAM	Freq (MHz)	Latency (ns)	Max Throughput (Gbps)
GCQ [6]	RTL	1024 (256)	16	✗	✗	RR-Only ¹	✓	60.0	13.5	224	180	172	160 (40.9)
SMiSLIP [8] ²	RTL	256	16	✗	✗	✓	✓	80.3	23.1	NA	140	41.4	35.8
		512	16	✗	✗	✓	✓	123.8	24.5	NA	110	44.3	56.3
Hipernetch [7] ²	RTL	256	16	✗	✗	✓	✗	150	125	NA	225	66	58
		512	16	✗	✗	✓	✗	200	300	NA	175	87	89
VitisNetP4 [37]	P4+RTL	256	-	✓	✓	✗	✗	5.21	8.99	9	259	176	66.3
P4-to-FPGA [39]	P4+RTL	272	-	✓	✓	✗	✗	5.64	1.94	2	176	40	48.1
P4THLS [38]	P4+HLS	256	-	✓	✓	RR-Only ¹	✗	5.97	3.58	4	250	40	69.5
SPAC Core-Only	HLS	256	-	✓	✓	RR-Only ¹	✗	4.47	7.01	2	350	34.3	89.6
SPAC Ethernet	HLS	512	8	✓	✓	✓	✓	80.1	45.8	304	146	68.3	74.7
			16	✓	✓	✓	✓	315.6	135.1	608	137	109.2	70.1
SPAC Basic	HLS	256	8	✓	✓	✓	✓	38.9	30.5	260	165	57.3	84.5
			16	✓	✓	✓	✓	96.1	83.2	498	142	85.5	72.7

¹ Only support Round-Robin scheduler.

² BRAM usage not provided.

- **Underwater:** Simulates underwater communication among 8 underwater robots using the DESERT [48] scheme, characterized by regular communication with minimal payloads ($\approx 2B$).

Baselines. Since most prior FPGA-based switch designs focus on individual modules (e.g., scheduler or crossbar only), direct end-to-end comparison with SPAC, which covers the complete switch pipeline (parser, forwarding table, scheduler, and VOQ buffer), is inherently limited. We therefore adopt **SPAC Ethernet**—Ethernet protocol with MultiBankHash, $N \times N$ VOQ, and iSLIP scheduling—as the default baseline, representing a general-purpose design point. We structure experiments into three levels: (1) unloaded datapath comparison (Table I, post-place-and-route); (2) scalability analysis under varying port counts (Figure 8); and (3) trace-driven DSE co-optimization (Table II, hardware-aware ns-3 simulation with cycle-level back-annotation).

B. Resource Efficiency & Scalability

1) *Resource Usage:* Table I reports unloaded datapath properties. Max Throughput is defined as $\text{datawidth} \times \Pi \times f_{\max}$; Latency refers to single-packet port-to-port traversal without contention. We synthesized three configurations for comparison: **SPAC Core-Only** retains only basic scheduling and parsing to isolate core datapath latency. **SPAC Ethernet** is the baseline (Section 6.1). **SPAC Basic** uses a compressed protocol for small-scale networks while keeping the same underlying architecture.

Most previous approaches use manually optimized RTL, offering deterministic latency at the cost of maintainability. GCQ achieves 160 Gbps theoretical throughput via grouping, but its single-bus bandwidth is only 40.9 Gbps (vs. our 70.1 Gbps), and clock domain crossing introduces 172ns latency— $1.57\times$ our unoptimized and $2.01\times$ our optimized design. SMiSLIP uses shared buffering for lower resource utilization, but its centralized scheduler limits throughput as port count grows. Hipernetch achieves high throughput via pipelined parallel round-robin arbiters, at the cost of significant resource consumption.

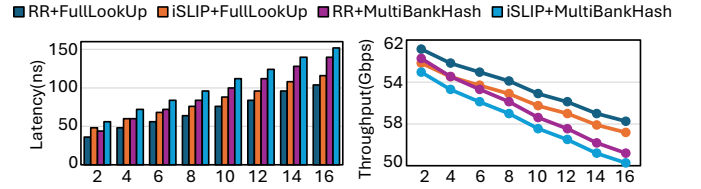


Fig. 8. Average P2P Performance Under Different #Port and Architectures.

Note that these prior designs implement *only* scheduler/crossbar modules, whereas SPAC includes the full-stack datapath (parsing, forwarding-table learning/lookup, scheduling, and buffering), which increases pipeline depth and end-to-end latency. Although the resource consumption of SPAC Ethernet and SPAC Basic is slightly higher than these works, we achieve performance comparable to manually tuned RTL using HLS. More importantly, we provide a flexible, configurable, full-stack switch architecture, including automatic look-up table learning and protocol adaptation, which is ignored by other works. To demonstrate the efficiency of our core design, we synthesized SPAC Core, retaining only the simplest scheduler and packet parsing logic, to compare against VitisNetP4, P4-to-FPGA and P4THLS, which also only include simple parsing and forwarding logic. Since the majority of SPAC’s resources are typically dedicated to organizing complex schedulers and VOQ buffers, our streamlined design achieves lower LUT consumption and a $1.4\sim 2.0\times$ frequency improvement, thereby yielding higher theoretical throughput.

2) *Scalability:* SPAC supports diverse combinations of switch architectures and port configurations. Due to the inherent non-determinism of HLS synthesis, latency and throughput fluctuate as the port count changes. We show SPAC Ethernet’s performance with medium-sized packets ($\approx 512B$). As shown in Figure 8, latency and throughput exhibit approximately linear trends as port counts rise. This behavior stems from the increasing complexity of scheduling decisions and the significant lookup delays introduced by the expansion of forwarding table capacity. Under high loads, complex combinatorial logic increases the pipeline initiation interval, degrading average throughput. The decline rate correlates with forwarding table

TABLE II
COMPARISON OF SWITCH DESIGN AND PERFORMANCE AFTER
OPTIMIZATION IN DIFFERENT APPLICATIONS

Application (Num nodes)	Architecture	Data Width (Bits)	Avg Header (Payload) (Bytes)	Avg Latency (ns)	Opt Baseline Latency (ns)	Latency Breakdown
HFT (8)	FullLookup N*N VOQs RR	256	2 (24)	64	103.9	38.4%
RL (8)	FullLookup N*N VOQs EDRRM	1024	2 (1463)	538	20391.7	97.3% [†]
Datacenter (32)	MultiBank Shared VOQs iSLIP	256	4 (965.5)	154.1	167.2	7.8%
Industry (10)	FullLookup Shared VOQs RR	128	2 (58.7)	76	119.9	36.6%
Underwater (8)	FullLookup Shared VOQs RR	256	2 (2)	42	68.3	38.5%

[†] Baseline experienced packet loss during incast.

architecture, as MultiBankHash bank conflicts increase with lookup traffic. Overall, our design achieves a latency of approximately 109ns at 16 ports, only 63.4% of the latency observed in the GCQ switch with identical specifications, demonstrating the performance of our HLS-implemented modules.

C. Domain Specific Adaption

We conducted experiments on customized switch architectures using various network flows with distinct characteristics. These flows represent scenarios characterized by high-frequency small packets, ultra-low latency requirements, high-throughput incast patterns, and large-scale sparse connectivity. Table II presents the switch architectures and packet statistics resulting from the DSE trade-offs.

For small-scale networks, SPAC reduces protocol overhead via header compression (14B to 2B), enabling the use of low-latency FullLookup forwarding tables. In small-packet environments like HFT, the algorithm selects pipelined Round-Robin scheduling to achieve higher frequency and maintain line-rate throughput, whereas iSLIP is preferred for DataCenter workloads to mitigate HoL blocking under mixed traffic, providing lower average latency. The RL workload tends to converge on EDRRM, as it effectively balances the handling of bursty traffic with latency constraints. For sensor networks such as underwater robots, we compress the packet to just 4B [36]. This allows for a minimalist architecture with simplified buffering and scheduling logic. Compared to the SPAC Ethernet baseline at the same 8-port scale, this reduces LUT and BRAM usage by $\sim 55\%$ and $\sim 53\%$ respectively, while lowering latency to only 42ns.

While both DataCenter and Industrial scenarios opted for Shared VOQs, the underlying motivations differ. In DataCenter environments, the network scale typically involves a large number of connected nodes (N). Implementing fully partitioned $N \times N$ VOQs results in quadratic resource complexity ($O(N^2)$), which imposes excessive pressure on limited FPGA BRAM resources. Unlike HFT scenarios that demand ultra-low, deterministic latency, DataCenter workloads generally

exhibit looser latency constraints, allowing them to tolerate the slight logic overhead of pointer management. Given that RPC traffic in data centers consists primarily of unevenly distributed mice flows [49], Shared VOQs provide superior buffer utilization efficiency to absorb bursts without the prohibitive resource costs of static partitioning.

These results reveal how traffic characteristics drive architectural choices. In small-packet latency-sensitive workloads such as HFT, frequency dominates over scheduling fairness, so DSE favors pipelined RR to maximize $f_{\max}$. Under bursty incast patterns like RL All-Reduce, buffering and scheduling become the dominant factors for tail latency beyond line-rate, leading DSE to select wider buses with EDRRM. At larger scale in DataCenter workloads, $O(N^2)$ VOQ resource pressure makes buffer organization the primary bottleneck, and DSE shifts to shared VOQs with iSLIP to balance HoL blocking mitigation against resource constraints.

Our DSE model also optimized the switch bus width. In most small-scale, high-frequency scenarios, a 256-bit bus width provides sufficient bandwidth, making further expansion a resource waste. However, for bandwidth-intensive scenarios such as RL training, wider bus widths yield significant throughput improvements. Notably, in RL All-Reduce burst tests, the baseline switch suffered severe packet loss during Incast events. In contrast, the optimized design targets Incast ports with increased buffer allocation, maintaining latency within a reasonable range. Overall, compared to fixed-architecture counterparts, the SPAC switch delivers average latency breakdown of 7.8%~38.4% across various tasks. For future work, implementing targeted HLS-based in-switch computing kernels (e.g., All-Reduce aggregation [46], [50]) could yield even greater performance gains.

VI. CONCLUSION

This paper introduces SPAC, a framework that automates the co-design of custom protocols and FPGA-based switch micro-architectures. We propose a unified DSL-driven workflow that integrates three key innovations: a modular HLS-based switch template allowing flexible composition of protocols and switch architectures; a trace-aware DSE engine for identifying pareto-optimal configurations; and a multi-granularity simulation system that enables hardware-aligned verification within the ns-3 network simulator. Experiments show that this domain-specific adaptation effectively optimizes performance for diverse workloads, achieving resource savings of 55% in constrained environments and latency reductions of 7.8%~38.4% compared to static baselines. These results demonstrate that automated application-specific customization provides a scalable and efficient path for next-generation network infrastructure.

ACKNOWLEDGEMENT

The support of the UK EPSRC (Grant EP/V028251/1, EP/S030069/1, EP/X036006/1), UKRI (Grant 256), KIAT, AMD and Broadcom is gratefully acknowledged.

REFERENCES

- [1] Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, and M. Yu, "HPCC: high precision congestion control," in *Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM 2019, Beijing, China, August 19-23, 2019*, J. Wu and W. Hall, Eds. ACM, 2019, pp. 44–58. [Online]. Available: <https://doi.org/10.1145/3341302.3342085>
- [2] N. McKeown, "The islip scheduling algorithm for input-queued switches," *IEEE/ACM Trans. Netw.*, vol. 7, no. 2, pp. 188–201, 1999. [Online]. Available: <https://doi.org/10.1109/90.769767>
- [3] Y. Li, S. Panwar, and H. J. Chao, "The dual round robin matching switch with exhaustive service," in *Workshop on High Performance Switching and Routing, Merging Optical and IP Technologie*. IEEE, 2002, pp. 58–63.
- [4] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: programming protocol-independent packet processors," *Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, 2014. [Online]. Available: <https://doi.org/10.1145/2656877.2656890>
- [5] nsnam, "ns-3." [Online]. Available: <https://www.nsnam.org/>
- [6] Z. Dai and J. Zhu, "Saturating the transceiver bandwidth: switch fabric design on FPGAs," in *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 67–76. [Online]. Available: <https://doi.org/10.1145/2145694.2145706>
- [7] P. Papaphilippou, J. Meng, N. Gebara, and W. Luk, "Hipernetch: High-performance FPGA network switch," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 15, no. 1, pp. 1–31, 2021.
- [8] J. Meng, N. Gebara, H.-C. Ng, P. Costa, and W. Luk, "Investigating the feasibility of FPGA-based network switches," in *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, vol. 2160. IEEE, 2019, pp. 218–226.
- [9] N. Zilberman, Y. Audzevich, G. A. Covington, and A. W. Moore, "Netfpga sume: Toward 100 gbps as research commodity," *IEEE Micro*, vol. 34, no. 5, pp. 32–41, 2014.
- [10] S. Denholm, K. H. Tsoi, P. Pietzuch, and W. Luk, "CusComNet: A customisable network for reconfigurable heterogeneous clusters," in *ASAP 2011 - 22nd IEEE International Conference on Application-specific Systems, Architectures and Processors*, 2011, pp. 9–16.
- [11] P. Papaphilippou, K. Sano, B. A. Adhi, and W. Luk, "Experimental survey of FPGA-based monolithic switches and a novel queue balancer," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1621–1634, 2023.
- [12] Xilinx, "OpenNIC," 2021, accessed 2026-03-30. [Online]. Available: <https://github.com/Xilinx/open-nic>
- [13] A. Forench, A. C. Snoeren, G. Porter, and G. Papen, "Corundum: An open-source 100-gbps nic," in *28th IEEE Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM 2020, Fayetteville, AR, USA, May 3-6, 2020*. IEEE, 2020, pp. 38–46. [Online]. Available: <https://doi.org/10.1109/FCCM48280.2020.00015>
- [14] R. D. G. Pacifico, L. F. D. S. Duarte, L. F. M. Vieira, B. Raghavan, J. A. M. Nacif, and M. A. M. Vieira, "ebpflow: A hardware/software platform to seamlessly offload network functions leveraging ebpf," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1319–1332, 2024. [Online]. Available: <https://doi.org/10.1109/TNET.2023.3318251>
- [15] Xilinx, "nanotube," 2023, accessed 2026-03-30. [Online]. Available: <https://github.com/Xilinx/nanotube>
- [16] J. Lin, K. Patel, B. E. Stephens, A. Sivaraman, and A. Akella, "PANIC: A high-performance programmable NIC for multi-tenant networks," in *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 2020, pp. 243–259. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/lin>
- [17] Z. Zhao, H. Sadok, N. Atre, J. C. Hoe, V. Sekar, and J. Sherry, "Achieving 100gbps intrusion prevention on a single server," in *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 2020, pp. 1083–1100. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/zhao-zhipeng>
- [18] S. Pontarelli, R. Bifulco, M. Bonola, C. Cascone, M. S. Brunella, V. Bruschi, D. Sanvito, G. Siracusano, A. Capone, M. Honda, and F. Huici, "Flowblaze: Stateful packet processing in hardware," in *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26-28, 2019*, J. R. Lorch and M. Yu, Eds. USENIX Association, 2019, pp. 531–548. [Online]. Available: <https://www.usenix.org/conference/nsdi19/presentation/pontarelli>
- [19] X. Chen, J. Zhang, T. Fu, Y. Shen, S. Ma, K. Qian, L. Zhu, C. Shi, Y. Zhang, M. Liu, and Z. Wang, "Demystifying datapath accelerator enhanced off-path smartnic," in *32nd IEEE International Conference on Network Protocols, ICNP 2024, Charleroi, Belgium, October 28-31, 2024*. IEEE, 2024, pp. 1–12. [Online]. Available: <https://doi.org/10.1109/ICNP61940.2024.10858560>
- [20] M. S. Brunella, G. Belocchi, M. Bonola, S. Pontarelli, G. Siracusano, G. Bianchi, A. Cammarano, A. Palumbo, L. Petrucci, and R. Bifulco, "hxdp: Efficient software packet processing on FPGA nics," *Commun. ACM*, vol. 65, no. 8, pp. 92–100, 2022. [Online]. Available: <https://doi.org/10.1145/3543668>
- [21] X. Li, X. Jiang, Y. Yang, L. Chen, Y. Wang, C. Wang, C. Xu, Y. Lv, B. Yang, T. Wu, H. Gao, Z. Chen, Y. Qiao, H. Ding, Y. Dong, H. Yang, J. Song, J. Lu, P. Zhang, C. Wei, Z. Zhang, W. Chen, Q. He, and S. Zhu, "Triton: A flexible hardware offloading architecture for accelerating apsara vswitch in alibaba cloud," in *Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024*. ACM, 2024, pp. 750–763. [Online]. Available: <https://doi.org/10.1145/3651890.3672224>
- [22] S. A. Fahmy, Z. Yang, Y. Chen, G. Alonso, Z. István, and M. Canini, "FPGAs are the hero in-network computing needs," in *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems*, 2025, pp. 131–139.
- [23] Y. Li, I.-J. Liu, Y. Yuan, D. Chen, A. Schwing, and J. Huang, "Accelerating distributed reinforcement learning with in-switch computing," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 279–291.
- [24] Y. Tokusashi, H. T. Dang, F. Pedone, R. Soulé, and N. Zilberman, "The case for in-network computing on demand," in *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019, pp. 1–16.
- [25] A. Sapio, I. Abdelaziz, A. Aldilajan, M. Canini, and P. Kalnis, "In-network computation is a dumb idea whose time has come," in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks*, 2017, pp. 150–156.
- [26] S. Kianpisheh and T. Taleb, "A survey on in-network computing: Programmable data plane and technology specific applications," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 701–761, 2022.
- [27] M. Nickel and D. Göhringer, "A survey on architectures, hardware acceleration and challenges for in-network computing," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 1, pp. 1–34, 2024.
- [28] "IEEE Standard for Ethernet," *IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018)*, pp. 1–7025, Jul. 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9844436>
- [29] "ietf.org/rfc/rfc793.txt." [Online]. Available: <https://www.ietf.org/rfc/rfc793.txt>
- [30] [Online]. Available: https://www.afs.enea.it/asantoro/V2r1_2_1_Release.pdf
- [31] FIX Trading Community, *FIX 5.0 Service Pack 2 Specification*, FIX Trading Community, 2011, financial Information eXchange Protocol. [Online]. Available: <https://www.fixtrading.org/standards/fix-5-0-sp-2/>
- [32] FIX Protocol Ltd, "FAST protocol," FIX Trading Community, Standard Specification, 2006, fix Adapted for STreaming. [Online]. Available: <https://www.fixtrading.org/standards/fast/>
- [33] "Industrial communication networks - fieldbus specifications - part 5-10: Application layer service definition - type 10 elements (profinet)," IEC, Standard Specification, 2023. [Online]. Available: <https://www.profinet.com/download/profinet-specification>
- [34] "Industrial communication networks - fieldbus specifications - part 5-10: Application layer service definition - type 12 elements (ethercat)," IEC, Standard Specification, 2023. [Online]. Available: https://www.ethercat.org/en/downloads/downloads_A02E436C7A97479F9261FDFA8A6D71E5.htm
- [35] M. Alizadeh, A. G. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in *Proceedings of the ACM SIGCOMM 2010 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, New Delhi, India, August 30 -September 3, 2010*, S. Kalyanaraman, V. N. Padmanabhan, K. K. Ramakrishnan, R. Shorey, and G. M. Voelker, Eds. ACM, 2010, pp. 63–74. [Online]. Available: <https://doi.org/10.1145/1851182.1851192>

- [36] A. Brahmakshatriya, C. Rinard, M. Ghobadi, and S. P. Amarasinghe, "Netblocks: Staging layouts for high-performance custom host network stacks," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, pp. 467–491, 2024. [Online]. Available: <https://doi.org/10.1145/3656396>
- [37] Xilinx, "Vitisnetp4: P4 language support for xilinx devices," <https://www.xilinx.com/products/intellectual-property/ef-di-vitisnetp4.html>, 2024, [Accessed 13-01-2026].
- [38] M. Abbasmollaei, T. Ould-Bachir, and Y. Savaria, "P4thls: A templated hls framework to automate efficient mapping of p4 data-plane applications to fpgas," *IEEE Access*, 2025.
- [39] Z. Cao, H. Su, Q. Yang, J. Shen, M. Wen, and C. Zhang, "P4 to fpga-a fast approach for generating efficient network processors," *IEEE Access*, vol. 8, pp. 23 440–23 456, 2020.
- [40] P. Benáček, V. Pus, and H. Kubátová, "P4-to-vhdl: Automatic generation of 100 gbps packet parsers," in *24th IEEE Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM 2016, Washington, DC, USA, May 1-3, 2016*. IEEE Computer Society, 2016, pp. 148–155. [Online]. Available: <https://doi.org/10.1109/FCCM.2016.46>
- [41] H. Wang, R. Soulé, H. T. Dang, K. S. Lee, V. Shrivastav, N. Foster, and H. Weatherspoon, "P4FPGA: A rapid prototyping framework for P4," in *Proceedings of the Symposium on SDN Research, SOSR 2017, Santa Clara, CA, USA, April 3-4, 2017*. ACM, 2017, pp. 122–135. [Online]. Available: <https://doi.org/10.1145/3050220.3050234>
- [42] S. Ibanez, G. J. Brebner, N. McKeown, and N. Zilberman, "The p4->netfpga workflow for line-rate packet processing," in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, FPGA 2019, Seaside, CA, USA, February 24-26, 2019*, K. Bazargan and S. Neuendorffer, Eds. ACM, 2019, pp. 1–9. [Online]. Available: <https://doi.org/10.1145/3289602.3293924>
- [43] J. Cabal, P. Benáček, L. Kekely, M. Kekely, V. Pus, and J. Korenek, "Configurable FPGA packet parser for terabit networks with guaranteed wire-speed throughput," in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, FPGA 2018, Monterey, CA, USA, February 25-27, 2018*, J. H. Anderson and K. Bazargan, Eds. ACM, 2018, pp. 249–258. [Online]. Available: <https://doi.org/10.1145/3174243.3174250>
- [44] B. Al-Duwairi, A. Shatnawi, A. Al-Hammouri, and M. Ababneh, "Dataset of scada traffic captures from a medical waste incinerator with injected cyberattacks," *Data in Brief*, vol. 63, p. 112294, 11 2025.
- [45] S. K. Balakrishnan, "Ai-defined flow control in programmable network fabric (ai-fabric): The nanosecond flow intelligence module (nfim) for ultra-low-latency scheduling," 11 2025.
- [46] Y. Li, I. Liu, Y. Yuan, D. Chen, A. G. Schwing, and J. Huang, "Accelerating distributed reinforcement learning with in-switch computing," in *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22-26, 2019*, S. B. Manne, H. C. Hunter, and E. R. Altman, Eds. ACM, 2019, pp. 279–291. [Online]. Available: <https://doi.org/10.1145/3307650.3322259>
- [47] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, Y. Ding, J. He, and C. Xu, "Characterizing microservice dependency and performance: Alibaba trace analysis," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 412–426.
- [48] R. Masiero, S. Azad, F. Favaro, M. Petrani, G. Toso, F. Guerra, P. Casari, and M. Zorzi, "Desert underwater: An ns-miracle-based framework to design, simulate, emulate and realize test-beds for underwater network protocols," in *2012 Oceans - Yeosu*, 2012, pp. 1–10.
- [49] A. Roy, H. Zeng, J. Bagga, G. Porter, and A. C. Snoeren, "Inside the social network's (datacenter) network," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM 2015, London, United Kingdom, August 17-21, 2015*, S. Uhlig, O. Maennel, B. Karp, and J. Padhye, Eds. ACM, 2015, pp. 123–137. [Online]. Available: <https://doi.org/10.1145/2785956.2787472>
- [50] A. Sapio, M. Canini, C. Ho, J. Nelson, P. Kalnis, C. Kim, A. Krishnamurthy, M. Moshref, D. R. K. Ports, and P. Richtárik, "Scaling distributed machine learning with in-network aggregation," in *18th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2021, April 12-14, 2021*, J. Mickens and R. Teixeira, Eds. USENIX Association, 2021, pp. 785–808. [Online]. Available: <https://www.usenix.org/conference/nsdi21/presentation/sapio>